CCP 프로그램 i.MX6 Linux 포팅 계획서

> *작성일: 2026-04-01 프로젝트: DENVER CCP (Central Control Panel) - WinCE → Linux 포팅 사유: 기존 CPU(S3C6410) 단종으로 인한 플랫폼 변경*

---

1. 현재 시스템 분석

1.1 기존 플랫폼 사양

항목	사양
CPU	Samsung S3C6410 (ARM11, 667MHz)
OS	Windows CE 6.0
개발환경	Visual Studio 2008, MFC
출력 바이너리	AnCCP_MFC.exe (약 797KB)
통신	RS-485 시리얼 (COM 포트)
디스플레이	터치 LCD (800x480 추정)
저장장치	NandFlash
DB	SQLite3

1.2 타겟 플랫폼 사양

항목	사양
CPU	NXP i.MX6 (ARM Cortex-A9)
OS	Linux (Yocto 기반)
개발환경	GCC 크로스 컴파일러 (arm-poky-linux-gnueabi-gcc)
빌드시스템	Makefile + Docker 크로스 컴파일
통신	/dev/ttymxc* (UART)
디스플레이	Linux Framebuffer (/dev/fb0)
저장장치	eMMC/NandFlash
DB	SQLite3 (동일)

1.3 소스코드 버전 현황

버전	경로	비고
v1.0.0.4 (최신)	`SW/엑스바이더/CCP_1.0.0.4 - 2016-01-31/`	포팅 기준 소스
v1.0.0.2	`SW/엑스바이더/CCP_1.0.0.2/`	이전 버전
GPS 버전	`SW/엑스바이더/CCP_1.0.0.4 - 2015-11-13(Gps).zip`	GPS 기능 포함
바이너리 2013	`SW/엑스바이더/CCP_Bin_20130604/`	배포 바이너리+리소스
바이너리 2012	`SW/엑스바이더/bin_20120619/`	초기 배포 바이너리
HW 회로도	`HW/아이텍/CCP-V1.4`	OrCAD 회로도+PCB

---

2. CCP 프로그램 기능 분석

2.1 화면(Page) 구성

Page ID	클래스	기능
HOME	CCCPDlg	메인 화면 - 열차번호, 승무원ID, 노선, 목적지, ETA, 지연시간, 현재역, 다음역, 장비상태 LED
TRAIN	CPageNumber	열차번호 입력 (6자리 숫자패드)
CREW	CPageNumber	승무원 ID 입력 (6자리 숫자패드)
ROUTE	CPageRoute	정차역 목록 표시, 급행/완행/통과 설정
CONFIG	CPageConfig	LCD 밝기, 스피커 볼륨, 시뮬레이션, C/I 설정, 시스템 리셋, 비밀번호
CHECK	CPageCar	차량 편성 상태 점검 (장비별 상태 표시)
HELP	CPageHelp	각 화면별 도움말 표시
PEI	CPageCar	승객 비상 인터콤 호출 표시
MESSAGE	CPageMessage	캡 텍스트 메시지
CM	CPagePR	프리셋 커스텀 메시지
CI	CPageCI	C/I (Consist Information) 편성 정보 설정

2.2 핵심 모듈 구성

```
CCP 프로그램 구조
├── 앱 프레임워크
│   ├── CCP.cpp/h           - MFC 앱 진입점 (CCCPApp)
│   ├── CCPDlg.cpp/h        - 메인 다이얼로그 (69KB, 핵심)
│   └── stdafx.h            - 프리컴파일 헤더, 매크로 정의
│
├── 통신 모듈
│   ├── Serial.cpp/h        - RS-485 시리얼 통신 스레드 (35KB)
│   └── Packet.cpp/h        - 패킷 파싱/생성, CRC16, BCD 변환 (15KB+25KB)
│
├── 데이터 모듈
│   ├── DBRoute.cpp/h       - 노선 DB 관리 (26KB)
│   ├── DBSchedule.cpp/h    - 스케줄 DB 관리
│   ├── DBTrigger.cpp/h     - 트리거 포인트 DB
│   ├── DBCM.cpp/h          - 커스텀 메시지 DB
│   ├── CeSQLite.cpp/h      - SQLite3 래퍼 클래스 (16KB)
│   └── StatusVariable.cpp/h - 상태 변수 관리 (11KB)
│
├── UI 모듈
│   ├── PageNumber.cpp/h    - 숫자 입력 페이지
│   ├── PageRoute.cpp/h     - 노선 페이지
│   ├── PageConfig.cpp/h    - 설정 페이지
│   ├── PageHelp.cpp/h      - 도움말 페이지
│   ├── PageCar.cpp/h       - 차량 점검/PEI 페이지 (27KB)
│   ├── PageCI.cpp/h        - C/I 설정 페이지
│   ├── PageMessage.cpp/h   - 메시지 페이지
│   ├── PagePR.cpp/h        - 프리셋 메시지 페이지
│   ├── PageManager.cpp/h   - 페이지 전환 관리
│   ├── BmpButton.cpp/h     - 비트맵 버튼 컨트롤
│   ├── BmpDC.cpp/h         - 비트맵 DC 관리
│   └── RichStatic.cpp/h    - 리치 텍스트 스태틱 컨트롤
│
├── 유틸리티
│   ├── Update.cpp/h        - 소프트웨어 업데이트 (14KB)
│   ├── LogThread.cpp       - 로그 기록 스레드
│   ├── util.cpp/h          - 유틸리티 함수
│   └── bmpfile.h           - BMP 파일 파싱
│
└── 리소스
    ├── RES/                - BMP 이미지 168개
```

```
    ├── CCP_DB/            - SQLite DB + 설정 파일
    ├── CONFIG.INI         - 설정 (TP_RADIUS = 40)
    ├── password.ini       - 비밀번호
    └── HELP.TXT           - 도움말 텍스트
```

2.3 통신 프로토콜

```
패킷 구조: [STX] [DATA (ASCII HEX)] [ETX]

DATA 구조:
  [SRC_ID:2] [DST_ID:2] [MSG_CNT:2] [CMD_CODE:2] [DATA_LEN:4] [PAYLOAD:N] [CRC16:4]

장비 ID:
  PAC-A: 0x21    PAC-B: 0x25
  CCP-A: 0x11    CCP-B: 0x12
  CNCS-A: 0x24   CNCS-B: 0x28
  LIC-A: 0x22    LIC-B: 0x26

명령 코드:
  0x10: INFO_REQ (정보 요청)
  0x11: INFO_ACK (정보 응답)
  0x12: INFO_VER (버전 정보)
  0x13: INFO_ACK_VER (버전 응답)
```

2.4 PAC 데이터 구조 (수신)

```
PAC 패킷 페이로드:
  PAC_T(1)  C_ID(1)  D0(1)  D1(1)  D2(1)
  ODM(4, BCD8)  ANS(1, BCD2)
  TRAN_NO(3, BCD6)  CREW_ID(3, BCD6)
  TRIP(3)  D3(1)
  STST(2, BCD4)  NOST(2, BCD4)  NEST(2, BCD4)  DEST(2, BCD4)
  SPK(2)  D4(1)  D5(1)  PR(1)
  PRVCT(2, BCD4)  PACVCT(2, BCD4)  CLRVCT(2, BCD4)
  CCI(1)  CI_LIC[8]  CI_USER[8]
  CAR_DATA[N] × 22바이트
```

2.5 차량 데이터 (CAR_DATA) 비트 매핑

```
BYTE 1: CABA, HEADA, CABB, HEADB
BYTE 2: WLRA, GPSA, VOIPCA, CCPA, WLRB, GPSB, VOIPCB, CCPB
BYTE 3: CNCSA, VTXA, LICA, PACA, CNCSB, VTXB, LICB, PACB
BYTE 4: PII2A, PII1A, FDIA, PII2B, PII1B, FDIB
BYTE 5: SDI4A~SDI1A, SDI4B~SDI1B
BYTE 6+: PID 데이터
```

---

3. 포팅 전략

3.1 레이어별 포팅 난이도

레이어	WinCE 원본	Linux 대체	난이도	작업량
비즈니스 로직	Packet, DBRoute, StatusVariable	순수 C로 변환	★☆☆☆☆	2주
데이터베이스	CeSQLite (sqlite3)	sqlite3 직접 사용	★☆☆☆☆	3일
시리얼 통신	Win32 COM API	POSIX termios	★★☆☆☆	1주
스레드/타이머	CWinThread, SetTimer	pthread, timer_create	★★☆☆☆	1주
GPIO/하드웨어	IOCTL (EXS 드라이버)	Linux sysfs/ioctl	★★★☆☆	1주
UI 렌더링	MFC Dialog + BmpDC	Linux Framebuffer	★★★★☆	4주

터치 입력	WM_LBUTTONDOWN	/dev/input/event*	★★★☆☆	1주
소프트웨어 업데이트	NandFlash 파일 복사	eMMC 파일 복사	★★☆☆☆	3일

3.2 API 매핑 테이블

WinCE / MFC API	Linux POSIX 대체
`CreateFile(COM*)`	`open("/dev/ttymxc*")`
`ReadFile / WriteFile`	`read() / write()`
`WaitCommEvent`	`select() / poll()`
`SetCommMask / PurgeComm`	`tcflush() / tcsetattr()`
`CWinThread`	`pthread_create()`
`CEvent`	`pthread_cond_t`
`CCriticalSection`	`pthread_mutex_t`
`SetTimer / OnTimer`	`timer_create() / signal`
`GetTickCount`	`clock_gettime(CLOCK_MONOTONIC)`
`Sleep(ms)`	`usleep(ms*1000)`
`CString`	`char[]` + snprintf
`CArray`	정적 배열 또는 동적 배열
`CFile`	`fopen / fread / fwrite`
`CreateDirectory`	`mkdir()`
`GetFileAttributes`	`stat()`
`CDialog / CWnd`	Framebuffer 직접 렌더링
`CDC / CBitmap`	Framebuffer 메모리 매핑
`RETAILMSG`	`printf / syslog`
`CreateMutex (중복실행방지)`	`flock()`
`DeviceIoControl (GPIO)`	`ioctl() / sysfs GPIO`
`DRVESC_SET_BRIGHTNESS`	sysfs backlight 제어
`TouchCalibrate()`	tslib 또는 evdev

3.3 UI 포팅 방식: Linux Framebuffer 직접 렌더링

선택 이유:

- 기존 BMP 리소스 168개를 그대로 활용 가능
- 외부 라이브러리 의존성 최소화 (임베디드 환경에 적합)
- TCMS 프로젝트에서 이미 OCT 라이브러리로 framebuffer 사용 중
- Qt 등 무거운 프레임워크 불필요

구현 방식:

```
/dev/fb0 → mmap() → 메모리 버퍼에 직접 픽셀 쓰기
/dev/input/event* → 터치 이벤트 읽기
BMP 파일 → bmpfile.h 파서로 로드 → framebuffer에 blit
```

---

4. 디렉토리 구조 (신규 프로젝트)

```
DENVER_CCP/
└── ccp_linux/                    ← 신규 생성
```

```
├── Makefile                    ← 빌드 설정
├── Dockerfile                  ← Docker 크로스 컴파일 환경
├── build.sh                    ← 빌드 스크립트
│
├── src/                        ← 소스 코드
│   ├── main.c                  ← 프로그램 진입점
│   ├── ccp_app.c/h             ← 앱 초기화/메인루프
│   │
│   ├── serial/                 ← 시리얼 통신
│   │   ├── serial.c/h          ← POSIX 시리얼 포트
│   │   └── serial_thread.c/h ← 수신 스레드
│   │
│   ├── protocol/               ← 통신 프로토콜
│   │   ├── packet.c/h          ← 패킷 파싱/생성 (CRC16, BCD)
│   │   └── protocol_def.h      ← 장비ID, 명령코드 정의
│   │
│   ├── data/                   ← 데이터 관리
│   │   ├── db_route.c/h        ← 노선 DB
│   │   ├── db_schedule.c/h     ← 스케줄 DB
│   │   ├── db_trigger.c/h      ← 트리거 포인트 DB
│   │   ├── db_cm.c/h           ← 커스텀 메시지 DB
│   │   └── status_var.c/h      ← 상태 변수
│   │
│   ├── ui/                     ← UI 렌더링
│   │   ├── framebuffer.c/h     ← FB 초기화/픽셀 조작
│   │   ├── bmp_loader.c/h      ← BMP 파일 로더
│   │   ├── touch_input.c/h     ← 터치 입력 처리
│   │   ├── ui_manager.c/h      ← 페이지 전환 관리
│   │   ├── ui_button.c/h       ← 버튼 위젯
│   │   ├── ui_text.c/h         ← 텍스트 렌더링
│   │   └── pages/              ← 각 화면
│   │       ├── page_home.c/h
│   │       ├── page_number.c/h
│   │       ├── page_route.c/h
│   │       ├── page_config.c/h
│   │       ├── page_car.c/h
│   │       ├── page_ci.c/h
│   │       ├── page_help.c/h
│   │       ├── page_message.c/h
│   │       └── page_pr.c/h
│   │
│   ├── hw/                     ← 하드웨어 제어
│   │   ├── gpio.c/h            ← GPIO (CABON키, CONDUCTER키)
│   │   ├── backlight.c/h       ← LCD 밝기 제어
│   │   └── watchdog.c/h        ← 워치독
│   │
│   ├── util/                   ← 유틸리티
│   │   ├── logger.c/h          ← 로그 기록
│   │   ├── update.c/h          ← SW 업데이트
│   │   ├── config.c/h          ← 설정 파일 관리
│   │   └── timer.c/h           ← 타이머 관리
│   │
│   └── platform/               ← 플랫폼 추상화
│       └── pal.h               ← 타입 정의, 매크로
│
├── lib/                        ← 외부 라이브러리
│   └── sqlite3/                ← SQLite3 소스
│       ├── sqlite3.c
│       └── sqlite3.h
│
├── res/                        ← 리소스 (기존 BMP 복사)
│   └── *.bmp
│
└── data/                       ← 데이터 파일
    ├── DVDB.dat                ← SQLite DB
```

```
        ├── CONFIG.INI
        ├── password.ini
        └── HELP.TXT
```

---

5. 단계별 작업 계획

Phase 1: 빌드 환경 구축 (1주)

#	작업	상세	산출물
1-1	Docker 크로스 컴파일 환경	TCMS 빌드 환경 기반으로 CCP용 Docker 이미지 생성	Dockerfile
1-2	Makefile 작성	소스 파일 목록, 컴파일 플래그, 링크 옵션	Makefile
1-3	플랫폼 추상화 헤더	타입 정의 (BYTE, UINT, BOOL 등), 매크로	pal.h
1-4	빌드 스크립트	Docker 기반 빌드 자동화	build.sh
1-5	빌드 검증	빈 main.c로 크로스 컴파일 성공 확인	ccp_main (ELF ARM)

Phase 2: 비즈니스 로직 포팅 (2주)

#	작업	원본 파일	포팅 파일	주요 변경점
2-1	패킷 처리	Packet.cpp/h	packet.c/h	CObject 제거, 순수 C 구조체+함수
2-2	CRC16	Packet.cpp 내	packet.c 내	변경 없음 (순수 C)
2-3	BCD 변환 매크로	Packet.h	protocol_def.h	변경 없음 (순수 매크로)
2-4	상태 변수	StatusVariable.cpp/h	status_var.c/h	CObject 제거, CString→char[], CArray→배열
2-5	SQLite 래퍼	CeSQLite.cpp/h	sqlite3 직접 사용	MFC 의존성 제거, 에러 처리 단순화
2-6	노선 DB	DBRoute.cpp/h	db_route.c/h	CArray→배열, CString→char[], CWnd 제거
2-7	스케줄 DB	DBSchedule.cpp/h	db_schedule.c/h	CObject 제거
2-8	트리거 DB	DBTrigger.cpp/h	db_trigger.c/h	CObject 제거
2-9	CM DB	DBCM.cpp/h	db_cm.c/h	CObject 제거
2-10	설정 파일	CONFIG.INI 파싱	config.c/h	신규 작성 (간단한 INI 파서)
2-11	단위 테스트	-	패킷 파싱, DB 조회 테스트	x86에서 테스트 가능

Phase 3: 시리얼 통신 포팅 (1주)

#	작업	원본	포팅	주요 변경점
3-1	시리얼 포트 열기/닫기	Serial::OpenPort()	serial_open/close()	CreateFile→open, DCB→termios
3-2	시리얼 읽기/쓰기	Serial::ReadPort/WritePort	serial_read/write()	ReadFile→read, WriteFile→write
3-3	수신 스레드	Serial::Run()	serial_thread()	CWinThread→pthread, WaitCommEvent→select()
3-4	STX/ETX 파싱	Serial::Run() 내	serial_thread() 내	로직 동일, API만 변경
3-5	PAC 데이터 처리	Serial::Run() 내	serial_thread() 내	로직 동일
3-6	CNCS 데이터 송신	Serial::SendToCNCS()	serial_send_cncs()	WriteFile→write
3-7	RS-485 RTS 제어	-	gpio로 RTS 핀 제어	i.MX6 GPIO 매핑 필요

3-8	통신 테스트	-	시리얼 루프백 테스트	실제 장비 또는 시뮬레이터

Phase 4: UI 프레임워크 구축 (2주)

#	작업	상세
4-1	Framebuffer 초기화	/dev/fb0 open, mmap, 해상도/BPP 설정
4-2	더블 버퍼링	백버퍼에 그린 후 한번에 복사 (깜빡임 방지)
4-3	BMP 로더	bmpfile.h 기반, BMP→픽셀 배열 변환
4-4	기본 그리기 함수	fill_rect, draw_bmp, draw_text
4-5	폰트 렌더링	비트맵 폰트 또는 간단한 벡터 폰트
4-6	터치 입력	/dev/input/event* 읽기, 좌표 변환
4-7	버튼 위젯	영역 정의, 터치 히트 테스트, 상태(UP/DN) 전환
4-8	페이지 관리자	페이지 전환, 활성 페이지 렌더링
4-9	타이머 시스템	주기적 화면 갱신 (상태 업데이트, 시계 등)

Phase 5: 화면 페이지 구현 (3주)

#	페이지	원본 크기	주요 기능
5-1	HOME	CCPDlg (69KB)	메인 화면, 상태 LED, 스케줄 표시, 메뉴 버튼 9개
5-2	TRAIN/CREW	PageNumber (6KB)	숫자패드 입력, 유효성 검증
5-3	ROUTE	PageRoute (9KB)	정차역 목록, 급행/완행/통과 버튼
5-4	CONFIG	PageConfig (16KB)	LCD 밝기, 스피커, 시뮬레이션, 비밀번호, 리셋
5-5	CHECK	PageCar (27KB)	차량 편성도, 장비별 상태 (가장 복잡한 화면)
5-6	CI	PageCI (9KB)	C/I 편성 정보 설정
5-7	HELP	PageHelp (9KB)	도움말 텍스트 표시
5-8	MESSAGE	PageMessage (9KB)	캡 텍스트 메시지
5-9	CM/PR	PagePR (5KB)	프리셋 메시지

Phase 6: 하드웨어 연동 (1주)

#	작업	상세
6-1	GPIO 매핑	i.MX6 GPIO 핀 번호 확인 (CABON키, CONDUCTER키, 주소 설정)
6-2	CABON/CONDUCTER 키	GPIO 입력 읽기 (EXS 드라이버 → Linux GPIO)
6-3	CCP ID 판별	주소 설정 GPIO로 CCP-A/B 판별
6-4	LCD 밝기 제어	sysfs backlight 인터페이스
6-5	워치독	/dev/watchdog 사용
6-6	터치 캘리브레이션	tslib 또는 자체 구현

Phase 7: 유틸리티 및 마무리 (1주)

#	작업	상세
7-1	로그 시스템	파일 기반 로그 (/data/LOG/CCP_XXXXXX.txt)
7-2	SW 업데이트	업데이트 파일 수신/적용 로직
7-3	중복 실행 방지	flock() 기반

7-4	시그널 핸들러	SIGTERM, SIGINT 처리 (정상 종료)
7-5	자동 시작	systemd 서비스 파일 또는 init 스크립트
7-6	통합 테스트	전체 기능 동작 확인

---

6. 파일별 포팅 매핑

원본 파일 (WinCE)	크기	포팅 파일 (Linux)	변환 방식
CCP.cpp	1.8KB	main.c, ccp_app.c	재작성
CCPDlg.cpp	69KB	ccp_app.c + page_home.c	분리 재작성
CCPDlg.h	5.6KB	ccp_app.h + page_home.h	분리 재작성
Serial.cpp	35KB	serial.c + serial_thread.c	API 변환
Serial.h	1.5KB	serial.h	API 변환
Packet.cpp	15KB	packet.c	CObject 제거
Packet.h	25KB	packet.h + protocol_def.h	매크로 분리
StatusVariable.cpp	11KB	status_var.c	CObject/CString 제거
StatusVariable.h	5.4KB	status_var.h	CObject/CString 제거
DBRoute.cpp	26KB	db_route.c	CArray/CString 제거
DBRoute.h	4.3KB	db_route.h	CArray/CString 제거
DBSchedule.cpp	2.3KB	db_schedule.c	CObject 제거
DBSchedule.h	3.4KB	db_schedule.h	CObject 제거
CeSQLite.cpp	16KB	sqlite3 직접 사용	래퍼 제거
CeSQLite.h	3.8KB	제거	sqlite3.h 직접 사용
PageNumber.cpp	5.6KB	page_number.c	UI 재작성
PageRoute.cpp	8.8KB	page_route.c	UI 재작성
PageConfig.cpp	16KB	page_config.c	UI 재작성
PageCar.cpp	27KB	page_car.c	UI 재작성
PageCI.cpp	9.5KB	page_ci.c	UI 재작성
PageHelp.cpp	9KB	page_help.c	UI 재작성
PageMessage.cpp	8.8KB	page_message.c	UI 재작성
PagePR.cpp	5.5KB	page_pr.c	UI 재작성
PageManager.cpp	2.5KB	ui_manager.c	재작성
BmpButton.cpp	7.8KB	ui_button.c	FB 기반 재작성
BmpDC.cpp	0.4KB	framebuffer.c에 통합	FB 기반 재작성
RichStatic.cpp	5.4KB	ui_text.c	FB 기반 재작성
Update.cpp	14KB	update.c	파일 API 변환
LogThread.cpp	2.8KB	logger.c	CFile→fopen
bmpfile.h	14KB	bmp_loader.c/h	거의 그대로
util.cpp	0.5KB	해당 모듈에 통합	-
stdafx.h	3.6KB	pal.h	매크로만 추출

---

7. 일정 요약

Phase	작업	기간	누적
1	빌드 환경 구축	1주	1주
2	비즈니스 로직 포팅	2주	3주
3	시리얼 통신 포팅	1주	4주
4	UI 프레임워크 구축	2주	6주
5	화면 페이지 구현	3주	9주
6	하드웨어 연동	1주	10주
7	유틸리티 및 마무리	1주	11주
-	통합 테스트 및 디버깅	2주	**13주 (약 3개월)**

---

8. 리스크 및 대응

리스크	영향	대응 방안
i.MX6 보드 HW 미확정	GPIO/시리얼 매핑 불가	Phase 6을 후순위로, 추상화 레이어로 분리
터치 LCD 해상도 변경	UI 좌표 전체 수정	해상도 매크로화, 비율 기반 좌표 계산
BMP 리소스 색상 깊이 차이	화면 깨짐	BMP 로더에서 16/24/32bpp 자동 변환
시리얼 타이밍 차이	통신 실패	버퍼 크기/타임아웃 조정 가능하도록 설계
SQLite DB 호환성	DB 열기 실패	동일 sqlite3 버전 사용, 기존 DVDB.dat 테스트
폰트 렌더링 품질	가독성 저하	비트맵 폰트 사전 생성 또는 freetype 사용

---

9. 빌드 환경 (Docker)

TCMS 프로젝트의 기존 Docker 환경을 활용:

```
# Docker 이미지 빌드
docker build --platform linux/amd64 -t ccp-build:arm -f Dockerfile .

# 컴파일
docker run --rm --platform linux/amd64 --user root --entrypoint /bin/bash \
  -v "$(pwd)":/src ccp-build:arm \
  -c "cd /src && CC=arm-poky-linux-gnueabi-gcc ARCH=arm make -j4"

# 결과물
# ccp_main (ELF 32-bit ARM executable)
```

---

10. 배포 구조 (타겟 보드)

```
/opt/ccp/                       ← 설치 경로
├── ccp_main                    ← 실행 파일
├── res/                        ← BMP 리소스
│   └── *.bmp (168개)
├── data/                       ← 데이터
│   ├── DVDB.dat
│   ├── CONFIG.INI
│   ├── password.ini
│   └── HELP.TXT
└── log/                        ← 로그 (런타임 생성)
    └── CCP_XXXXXX.txt
```

11. 체크리스트

Phase 1 완료 조건

- ☐ Docker 이미지 빌드 성공
- ☐ ARM 크로스 컴파일 성공 (빈 main.c)
- ☐ pal.h 타입 정의 완료

Phase 2 완료 조건

- ☐ packet.c - 패킷 파싱/생성, CRC16 검증 통과
- ☐ db_route.c - DVDB.dat에서 노선 조회 성공
- ☐ status_var.c - 상태 변수 읽기/쓰기 동작

Phase 3 완료 조건

- ☐ 시리얼 포트 열기/닫기 성공
- ☐ STX/ETX 패킷 수신 파싱 성공
- ☐ PAC 데이터 수신 및 디코딩 성공

Phase 4 완료 조건

- ☐ Framebuffer에 BMP 이미지 표시 성공
- ☐ 터치 입력 좌표 읽기 성공
- ☐ 버튼 터치 → 콜백 동작

Phase 5 완료 조건

- ☐ 전체 9개 페이지 화면 표시
- ☐ 페이지 전환 동작
- ☐ 숫자패드 입력 동작
- ☐ 차량 편성도 표시

Phase 6 완료 조건

- ☐ CABON/CONDUCTER 키 감지
- ☐ CCP-A/B ID 판별
- ☐ LCD 밝기 조절

Phase 7 완료 조건

- ☐ 로그 파일 생성/기록
- ☐ 자동 시작 동작
- ☐ 전체 기능 통합 테스트 통과